I mille modi di creare una applicazione reattiva su Azure

1nn0va community
TD SYNNEX
UNIPORDENONE
moltiplica il valore della formazione

UNIPORDENONE
moltiplica il valore della formazione

TD SYNNEX

Un'app reattiva non è solo "real-time UI"

1 Event-driven

Reagisce a eventi, non richiede polling. Il sistema risponde quando qualcosa accade.

2 Scalabile sotto carico

Ogni componente scala in modo indipendente. Le risorse si adattano al traffico reale.

3 Resiliente

Backpressure, retry, dead-letter. Non perde messaggi e degrada in modo gestito.

4 Osservabile

Metriche, tracing, logging distribuito: sai sempre dove il sistema è, e perché.

I NUMERI CHE CONTANO

Perché investire sulla reattività?

Gli utenti si aspettano esperienze live. Le architetture reattive cambiano la percezione del prodotto e la produttività del team.

< 1s

latenza percepita come "istantanea" dall'utente

+35%

retention media su esperienze real-time vs polling

-70%

richieste HTTP eliminate dal server quando si passa a push

4x

throughput ottenibile con code + worker vs chiamate sincrone

La scelta "reattivo" non è tecnica: è un patto con l'utente

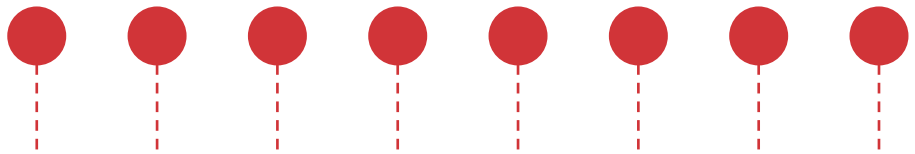
Ogni interazione in meno, ogni aggiornamento in più, ogni secondo risparmiato: è la differenza tra un'app che l'utente ama e una che sopporta.

CONFRONTO

Polling vs Eventi — due mondi, due costi

POLLING

Il client chiede, il server risponde (anche quando non c'è nulla di nuovo).

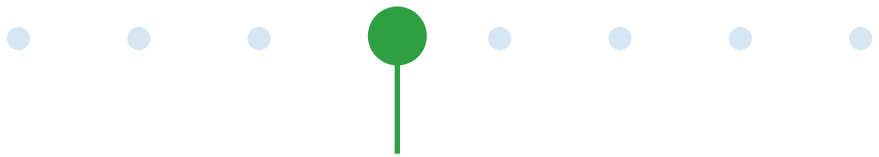


→ 8 richieste, 1 aggiornamento utile

- **Costo banda**
- **Carico inutile sul server**
- **Latenza = intervallo di polling**
- **Batteria consumata su mobile**

EVENTI

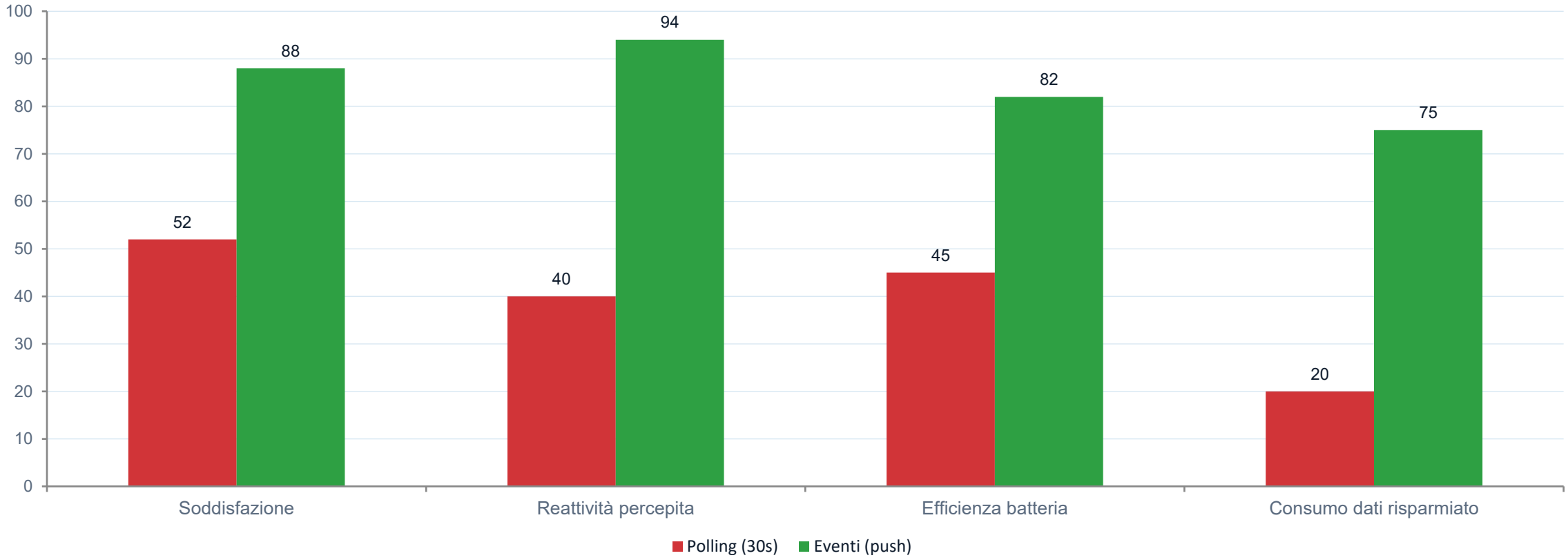
Il server notifica il client solo quando serve, al momento giusto.



→ 1 messaggio, 1 aggiornamento utile

- **Banda minima**
- **Carico proporzionale agli eventi reali**
- **Latenza in millisecondi**
- **Amico della batteria**

Soddisfazione utente • eventi vs polling



NON SOLO BROWSER

Reattività ovunque • web, mobile, desktop

Il pattern event-driven non è un problema di frontend. Ogni canale può diventare reattivo, con lo stesso backend.

WEB

SignalR JS client, React/Angular/Blazor. WebSocket nativo, fallback SSE/long-polling automatico.

`@microsoft/signalr`

MOBILE

Xamarin/MAUI, Flutter via interop, native iOS/Android tramite REST + WebSocket. Attenzione al cycle di background.

`Microsoft.AspNetCore.SignalR.Client`

DESKTOP

WPF, WinUI, Avalonia, Electron. Ideale per dashboard operative, trading, monitoring industriale.

`.NET client SignalR`

SERVER-TO-SERVER

Backend → backend: Service Bus + Functions coprono scenari di orchestration senza SignalR.

`Service Bus SDK`

Azure SignalR Service

Servizio managed per WebSocket/SSE/long-polling. Separa il canale real-time dal backend applicativo: scala, fallback e sicurezza sono gestiti dalla piattaforma.

Connessioni gestite

Terminazione WebSocket sul servizio; il tuo backend parla HTTP.

Fallback automatico

WebSocket → SSE → Long Polling senza che tu scriva una riga.

Scalabilità orizzontale

Unità scalabili da 1 a 100 — fino a 1M di connessioni concorrenti.

Serverless mode

Funziona con Azure Functions senza server di backend dedicato.

Integrazione Entra ID

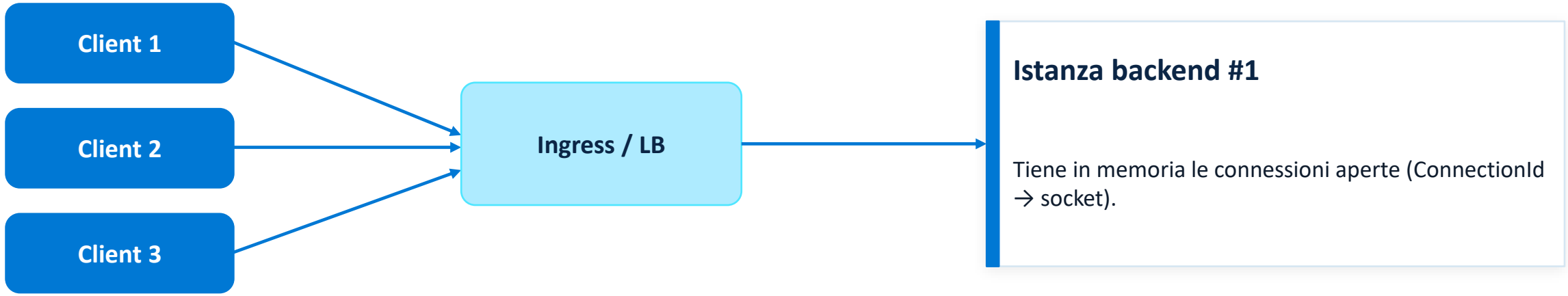
Auth con Microsoft Entra, chiavi di accesso, network private endpoint.

Modalità ASP.NET Core + ASP.NET classico

Supporta entrambi gli stack, stesso servizio.

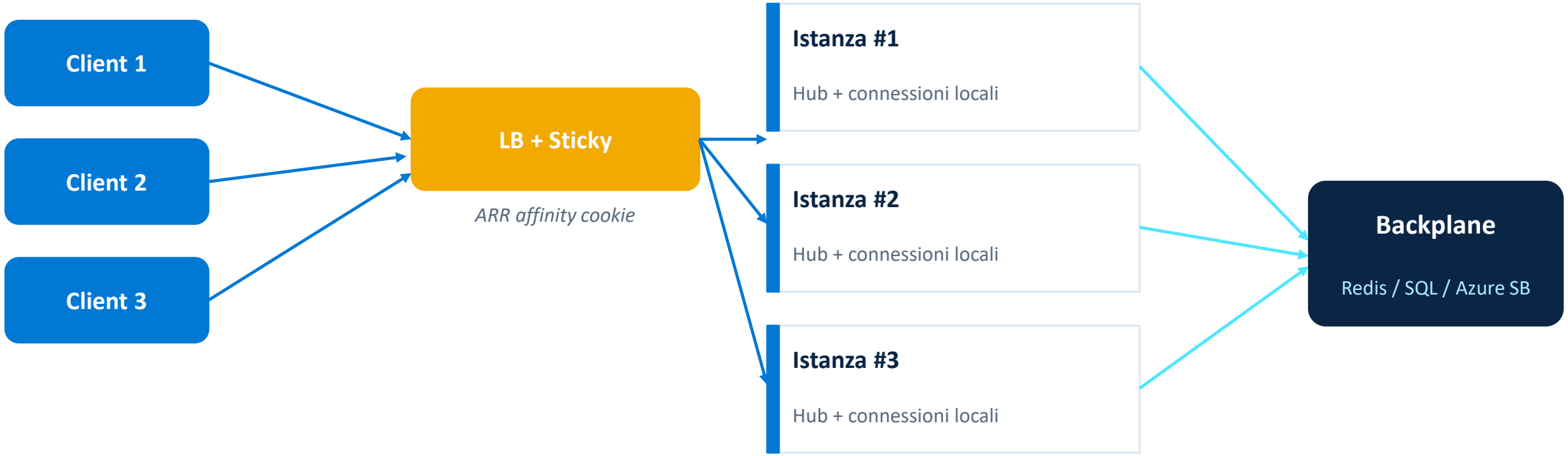
Sticky session — cosa succede su una sola istanza

Con una sola istanza di server, la sticky session è banale: c'è un solo nodo, tutti i client parlano con lui. Il problema nasce quando scali orizzontalmente.



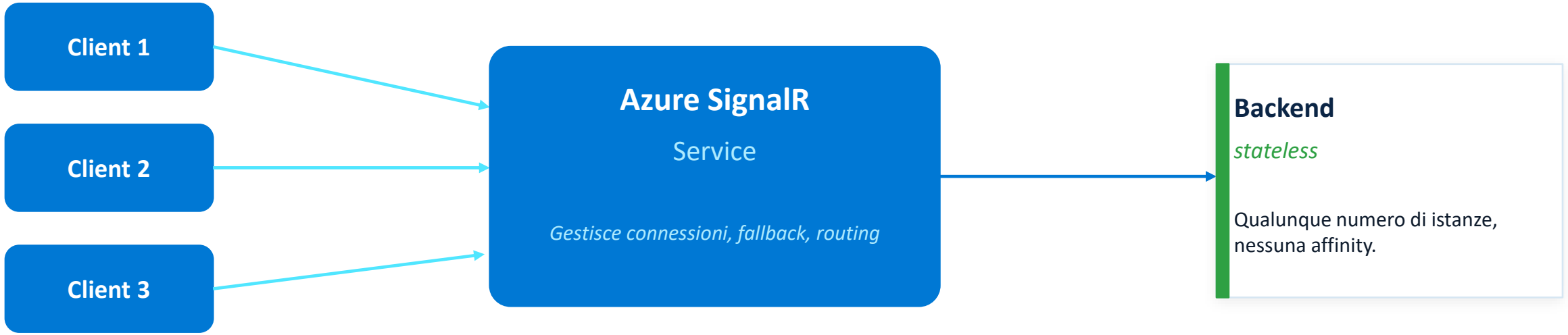
Sticky session: ogni client deve tornare sempre sulla stessa istanza. Con 1 nodo è gratis. Con N nodi, senza sticky, la connessione si rompe.

On-premise / multi-istanza — sticky + backplane



Trade-off: sticky rende il LB meno elastico. Il backplane aggiunge un SPOF e latenza. Con Azure SignalR entrambi i problemi spariscono.

Azure SignalR — niente sticky, niente backplane



1M
conn. max
(100 unit × 10k)

0
backplane
da gestire

Auto
fallback
WS → SSE → LP

Serverless
con Azure
Functions

Azure SignalR — SKU e limiti numerici

SKU / Tier	Unità	Conn. concorrenti / unit	Messaggi / unit / giorno	Uso tipico
Free	1 (fissa)	20	20.000	Prototipi, demo, dev
Standard	fino a 100	1.000	1.000.000	Produzione (auto/manual scale)
Premium	fino a 100	1.000	1.000.000	Geo-replica, SLA 99,95%, private endpoint

- **Max totale Standard / Premium** — 100 unit × 1.000 conn = 100.000 conn per risorsa • più risorse in parallelo per ~1M+
- **Message size** — 2 KB per messaggio (soft limit, si può scomporre)

Azure SignalR — quanto costa davvero

Tier	Costo unità / giorno	Costo mensile (1 unit)	Messaggi extra	Note
Free	0 €	0 €	non disponibile	20 conn, 20k msg/giorno
Standard	~ 1,50 €	~ 45 €	~ 1 € / milione msg extra	SLA 99,9%, auto-scale manuale
Premium	~ 3,00 €	~ 90 €	incluso pacchetto base	SLA 99,95%, geo-replica, VNet

Scenari di spesa (indicativi)

Prototipo
0 €

Startup
~45 €/mese

Scale-up
~450 €/mese

Enterprise
~3.000 €/mese

Azure Service Bus — non serve essere "grandi"

Non è un prodotto solo per enterprise. Anche un progetto piccolo guadagna tantissimo da una coda ben progettata: disaccoppiamento, retry, pattern async.

Disaccoppia producer e consumer

Chi produce non aspetta chi consuma. Il sistema resta reattivo anche sotto picco.

Retry e Dead-Letter nativi

Se un messaggio fallisce, viene reinviato con backoff. Dopo N tentativi finisce in DLQ, auditabile.

FIFO con Session

Ordine garantito per "conversation" (es. per utente, per ordine, per device).

Scheduling nativo

Invio differito di messaggi (es. reminder fra 1h) senza scrivere uno scheduler tuo.

Topic + Subscription (pub/sub)

Un evento, N consumer diversi con filtri declarativi. Cuore dei pattern event-driven.

Transazioni

Batching e transazioni tra queue/topic della stessa namespace.

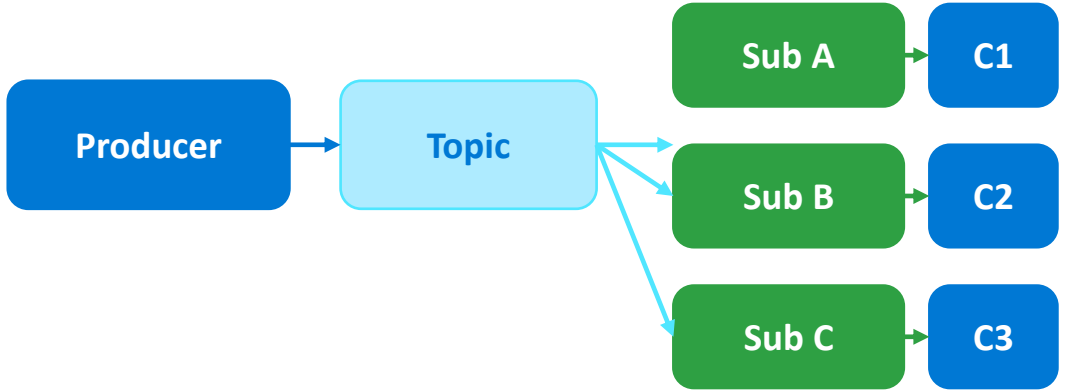
Queue vs Topic — scegli lo strumento giusto

Queue • 1 producer → N consumer (competing)

Ogni messaggio va a UN solo consumer. Il consumer più veloce "vince". Perfetto per load-balancing di job.

Usa la Queue per: job asincroni, elaborazioni batch, command pattern.

Topic • pub/sub — 1 producer → N subscriber



Ogni subscription riceve una COPIA del messaggio. Filtri su proprietà applicativi. Ideale per eventi di dominio.

Usa il Topic per: integrazione cross-bounded-context, event sourcing, fan-out.

Event-driven per il cliente — cosa cambia davvero

PRO

Esperienza in tempo reale
L'utente vede cambiamenti senza ricaricare. Percezione di "prodotto vivo".

Meno errori di coordinamento
Due operatori non lavorano sullo stesso record senza saperlo.

Meno richieste di assistenza
"È già aggiornato", "ho già visto" — meno attriti, meno ticket.

Competitive edge
Esperienze reattive diventano lo standard atteso (banking, e-commerce, logistica).

CONTRO

Maggiore complessità mentale
Il cliente si abitua alla velocità. Una latenza improvvisa diventa "bug" percepito.

Debug lato utente più difficile
"L'ho visto cambiare ma poi è tornato indietro" — servono audit trail chiari.

Costi infrastruttura leggermente più alti
SignalR + Service Bus + Functions introducono voci in più (ma scalano meglio).

Aspettative crescenti
Quando tutto è real-time, il cliente si aspetta che TUTTO sia real-time.

Event-driven per il team di sviluppo

PRO

Deploy indipendenti
Producer e consumer si deployano senza coordinarsi. Merge hell in meno.

Resilienza "gratuita"
Se un servizio è giù, la coda accumula. Quando torna, riprende da dove aveva lasciato.

Test più semplici
Un handler = una funzione pura su un evento. Test unitari senza mock di HTTP.

Evoluzione graduale
Aggiungi nuovi consumer senza toccare chi esisteva prima.

CONTRO

Eventual consistency
Non è "dopo la POST lo vedo subito": serve educare il team e la UX.

Debug distribuito
Serve observability seria (App Insights + correlation id) dal giorno 1.

Idempotenza obbligatoria
Ogni handler deve tollerare duplicati. Un messaggio prima o poi arriva due volte.

Curva di apprendimento
Dev junior hanno bisogno di mentorship: pattern, schema, naming di eventi.

Service Bus — SKU, limiti, message size

Tier	Queue	Topic / Sub	Max msg	Messaging Units	Throughput
Basic	Sì	No	256 KB	— (shared)	basso
Standard	Sì	Sì (2.000 sub/topic)	256 KB	— (shared)	medio, variabile
Premium	Sì	Sì (2.000 sub/topic)	100 MB	1, 2, 4, 8, 16	prevedibile e isolato

Namespace / sub
1.000

Queue / namespace (Std)
10.000

Queue / namespace (Prem)
1.000 per MU

TTL messaggio
fino a 14 giorni

Dimensione namespace (Std)
80 GB

Session massime
illimitato (con MU adeguate)

Service Bus — costi e soglie di scelta

Tier	Base	Scaglione operazioni	Scenario consigliato
Basic	~ 0 €, pay-per-op	~ 0,045 € / milione di op	Code semplici, poche op / mese
Standard	~ 9 € / mese namespace	~ 0,73 € / milione di op	Topic, filtri, traffico medio
Premium	~ 620 € / MU / mese	incluso	Latenza garantita, VNet, isolamento

Come scegliere in 10 secondi

Solo code, volumi bassi?

→ **Basic**

Topic o > 10 M op / mese?

→ **Standard**

Compliance, VNet, isolamento, grandi payload?

→ **Premium**

Azure Functions — il collante event-driven

Piccole unità di codice che reagiscono a un trigger: messaggio, timer, HTTP, evento. Perfette per integrare SignalR e Service Bus.

HTTP
Webhook, REST

Service Bus
Queue & Topic

Timer
CRON schedulati

Event Grid
Eventi Azure nativi

Blob Storage
File caricati

SignalR (output)
Push verso client

Pattern ricorrente • Service Bus trigger → elaborazione → SignalR output binding = push al client in 1 Function

Functions — piani di hosting, limiti a confronto

Piano	Timeout	Memoria	Scaling	Cold start
Consumption	5 min (max 10)	1,5 GB	event-driven, 0→200 istanze	Sì
Flex Consumption	fino a 30 min	2 / 4 GB	fast-scale, VNet, always-ready	ridotto
Premium	illimitato*	fino a 14 GB	pre-warmed, VNet, hybrid conn.	no (warm)
Dedicated (App Service)	illimitato	piano App Service	manuale/auto sul plan	no

200
istanze max • Consumption

100
istanze • Premium (soft)

60s
HTTP timeout su Consumption

10 min
timeout max singola Function (Consumption)

App Service — le feature che (quasi) tutti usano

Deployment Slots

Staging → swap a produzione senza downtime. Warm-up incluso.

Auto-scale

Regole su CPU, RAM, coda, metriche custom. Scale-in e scale-out automatici.

Continuous Deployment

GitHub Actions, Azure DevOps, Bitbucket — pipeline managed.

Custom Domains + SSL

Certificati gratuiti App Service Managed, binding automatico.

App Service Authentication

EasyAuth: Entra ID, Google, Facebook — senza scrivere codice auth.

Backup / Restore

Snapshot on-demand o pianificati di app + DB associato.

App Service — i tesori nascosti

1 Auto-Heal

Regole declarative: "se HTTP 500 > 5% per 2 min → riavvia". Senza tool esterni.

2 Hybrid Connections

Accesso a endpoint on-prem via Service Bus Relay, senza VPN o VNet.

3 Regional VNet Integration

App outbound in VNet privata — database interni accessibili, egress filtrabile.

4 Private Endpoint

App raggiungibile SOLO via IP privato, zero esposizione internet.

5 WebJobs

Job batch in-process con la web app: perfetti per background task.

6 Diagnose & Solve Problems

AI built-in che analizza log e metriche e suggerisce cause e fix.

7 Traffic Manager / Front Door

Geo-routing + WAF davanti a più App Service in regioni diverse.

8 App Service Environment

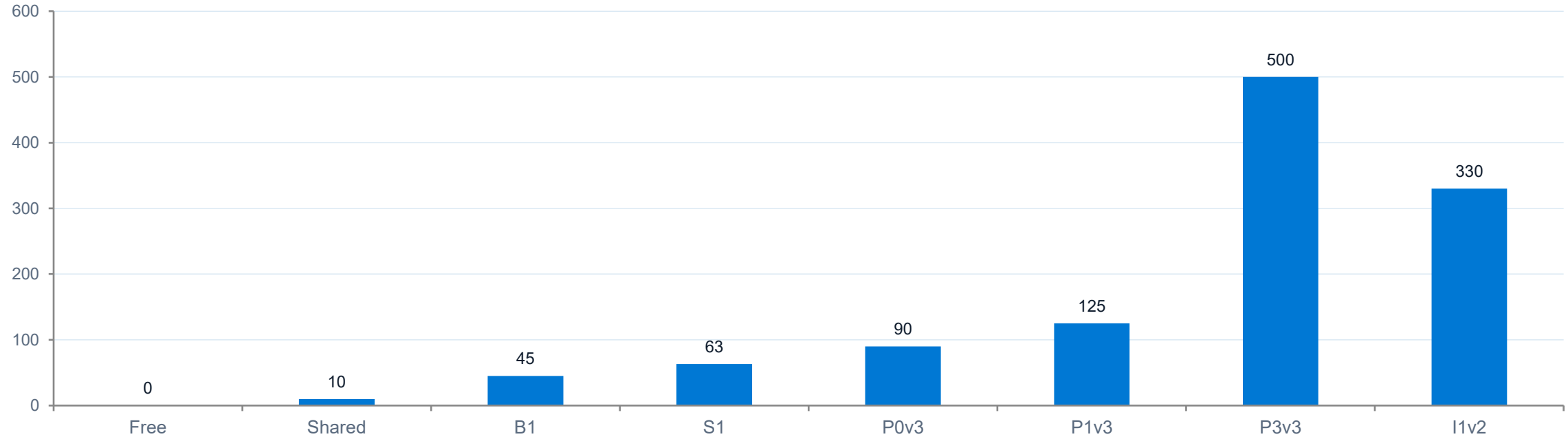
Multi-tenant isolato: la tua nuvola dentro la nuvola, per compliance.

App Service — SKU, istanze, limiti

Tier	vCPU / RAM per istanza	Istanze max	Slot	VNet / PE	Uso tipico
Free / Shared	shared	1	0	no	Demo, hobby
Basic (B1-B3)	1-4 vCPU / 1,75-7 GB	3	0	no	Dev / test
Standard (S1-S3)	1-4 vCPU / 1,75-7 GB	10	5	limitata	Produzione small
Premium v3 (P0-P5)	1-32 vCPU / 4-128 GB	30	20	sì	Produzione enterprise
Isolated v2 (ASE)	2-32 vCPU / 8-256 GB	100 per ASE	20	dedicato	Compliance, isolamento

Per piani auto-scale: regole su CPU/RAM/metriche custom, con scale-in controllato. • SLA: 99,95% dal tier Standard in su.

App Service — costo indicativo mensile



+ Slot

Gratis sullo stesso plan — usali sempre per blue/green.

+ Always On

Incluso dal tier Basic in su. Imposta sempre true.

+ Auto-scale

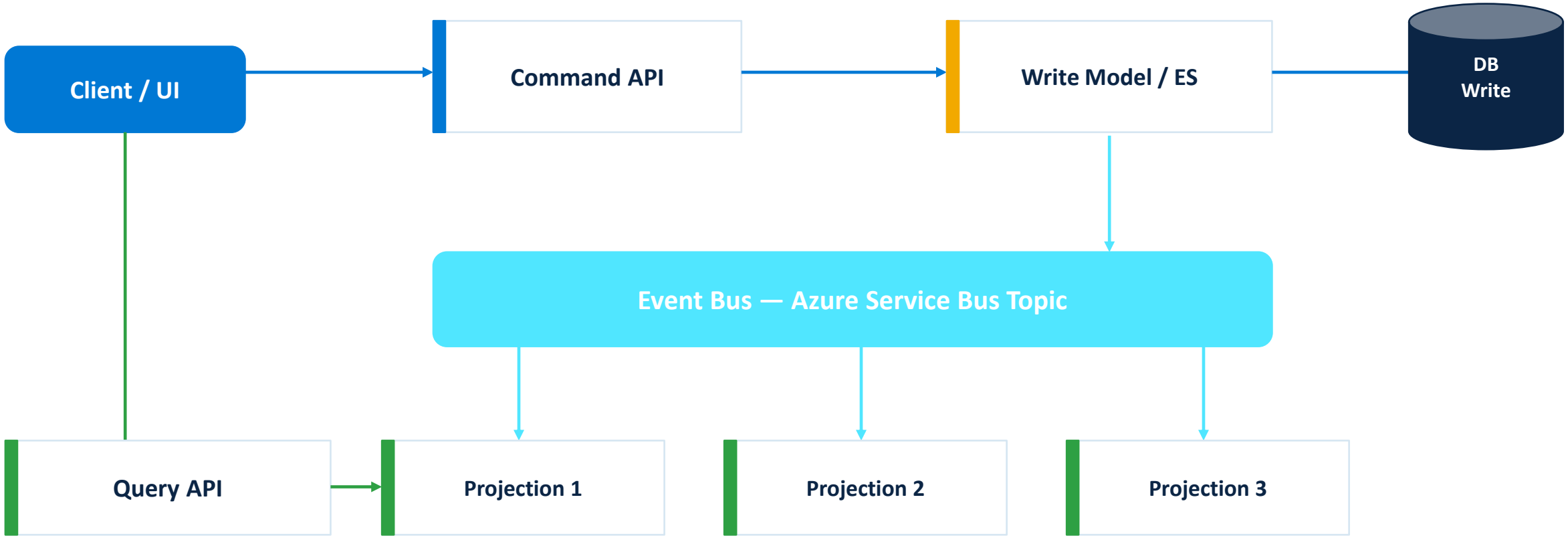
Solo Standard+: scale-out orizzontale fino al max del tier.

+ Reserved Instances

Piani 1-3 anni: -30/-55% sul costo annuo.

CQRS — due strade, due ottimizzazioni

Command Query Responsibility Segregation: separa il "scrivere" dal "leggere". Due modelli diversi, ottimizzati per scopi diversi.



Write model: normalizzato, transazionale, source of truth. • Read model: denormalizzato, per vista, cache-friendly, aggiornato via eventi.

Read model & aggregazione — performance e resilienza

Perché un read model dedicato

Una sola query = 1 round-trip
Documenti già assemblati per la vista — niente JOIN pesanti.

Scala indipendentemente dal write
Picchi di lettura non rallentano le scritture.

Ricostruibile dai eventi
Cambio di schema? Re-project. Non migri dati, li ricostruisci.

Storage giusto per lo scopo
Cosmos DB, Redis, Search, SQL: ogni vista nel suo posto.

Pipeline di aggregazione

1. **Event**
   OrderPlaced, PaymentReceived…
2. **Handler**
   Arricchisce, aggrega, deduplica
3. **Read Store**
   Cosmos / Redis / Search
4. **SignalR push**
   UI aggiornata in < 1s

Riepilogo • 5 idee da portare a casa

01	**Reattivo = patto con l'utente**	Prima di scegliere lo stack, chiediti quale esperienza vuoi dare. Il resto è conseguenza.
02	**SignalR toglie il problema sticky**	Su una istanza è gratis, con Azure SignalR è gratis pure a 1M di connessioni.
03	**Service Bus è per tutti**	Anche l'app da 3 microservizi guadagna disaccoppiamento, retry e DLQ nativi.
04	**CQRS è un pattern, non un prodotto**	Due modelli, due storage, eventi che li tengono allineati. Semplice, non banale.
05	**Il costo scala con il valore**	Parti Consumption / Free, migra ai tier Premium quando i numeri reali lo giustificano.

HANDS-ON

Repository, demo e risorse

Repository della sessione

github.com/andreatosato/GlobalAzure2026

Percorso: /src • demo complete di SignalR + Service Bus + Functions + CQRS

Azure SignalR Docs

learn.microsoft.com/azure/azure-signalr

Service Bus Docs

learn.microsoft.com/azure/service-bus-messaging

Azure Functions Docs

learn.microsoft.com/azure/azure-functions

App Service Docs

learn.microsoft.com/azure/app-service

DEMO